

Smart Home Demo Lab

Final Project Report

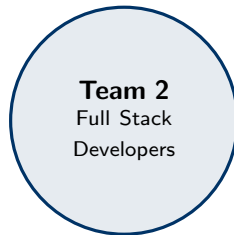
Yixue Wang, Linghao Dong, Yang Xiao, Chenxu Liu, Tong Lu

Freie Universität Berlin

February 2026

Team Members

- Yixue Wang
- Linghao Dong
- Yang Xiao
- Chenxu Liu
- Tong Lu



Topic & Project Motivation

Smart Home Demo Lab - Team 2

Topic

Programmable Scene: Define Automation Logic.

Project Motivation

- **Home Assistant:** Feature-heavy but overly complex.
- **Our Mission:** Simplify complexity while maintaining reliability.

Internal Goals / Requirements

- **Domain Driven Design (DDD) + Onion Architecture:** Independence
- **Extensibility:** Integration capabilities for new device and scenarios.
- **Testability:** Support for hardware mocking.

External Goals / Requirements

- **Integration:** Interact with Home Assistant API.
- **Automation:** Trigger-Condition-Action model.
- **Usability:** Graphical Scene Editor (based on DAG style).

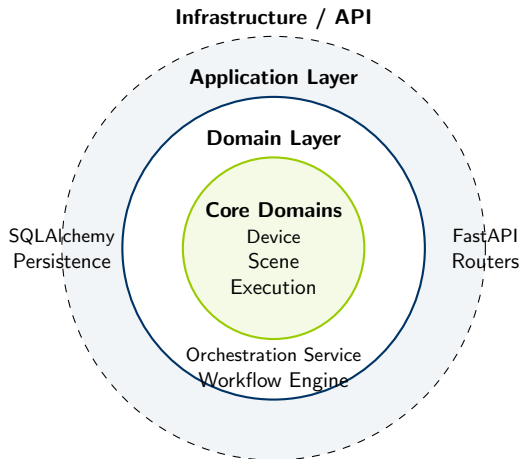
Minimum Viable Product (MVP)

Smart Home Demo Lab - Team 2

- **Device Management:** Device Registration, Status Monitoring and Control
- **Scenario Management:** Creation, save, and execution.
- **Automation Core:** Implementation of trigger, condition, action logic.
- **Frontend:** Interactive Visual Dashboard.

Implementation: Global Architecture

Strict adherence to Onion Architecture for maintainability



Benefit: Business logic is isolated from external tools (Database, HTTP API).

Implementation: Technology Stack

Modern, high-performance tooling

Backend

- **Language:** Python 3.10+
- **Framework:** FastAPI (Async)
- **Validation:** Pydantic v2
- **ORM:** SQLAlchemy 2.0

Frontend

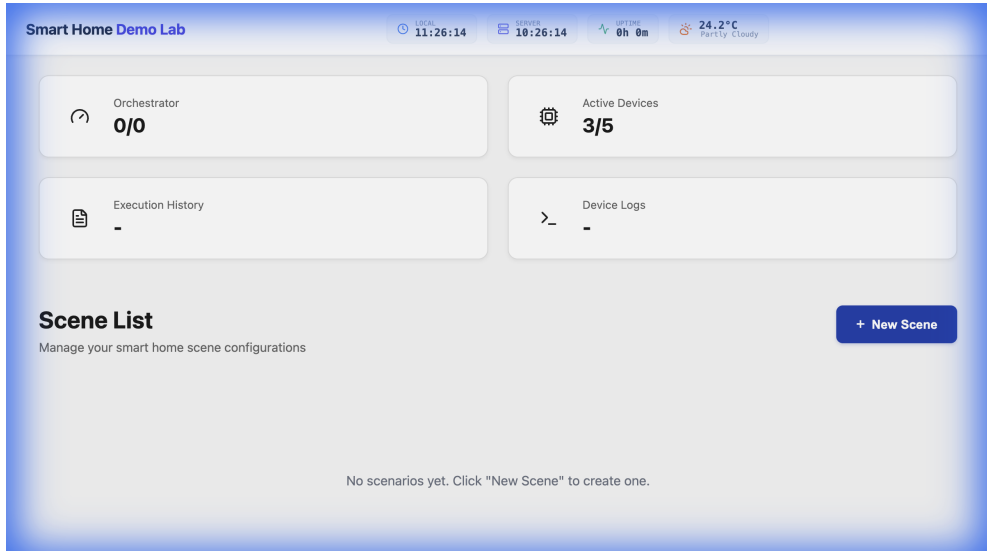
- **Framework:** React (Vite)
- **UI Lib:** TailwindCSS
- **Editor:** Custom DAG (Reference: n8n)

Infrastructure

- **Ecosystem:** Home Assistant (via API)
- **Mock Server:** Custom Python simulated hardware
- **Dev:** Docker & Pytest

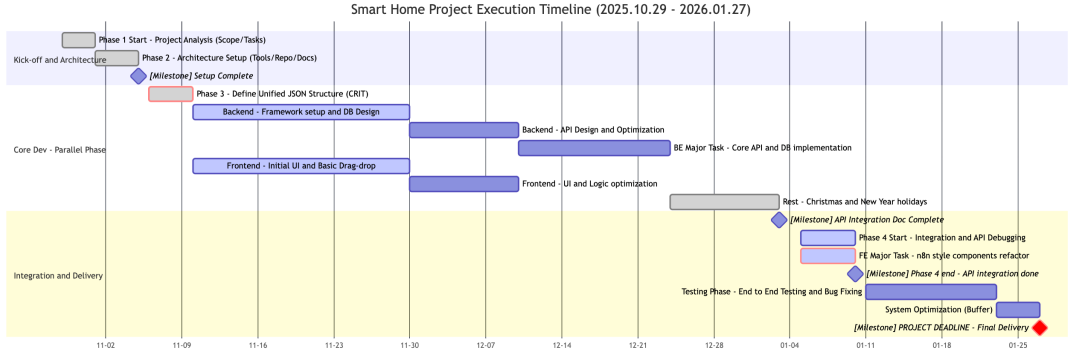
Solution: User Interface

Visualization of the Smart Home Controller Dashboard



Project Timeline

From Planning to Final Delivery



What did we achieve?

- Full-Stack Implementation
- Visual DAG Editor
- Hardware Simulation
- Complex Logic Solved

What conclusions did we reach?

- Visualization is the only solution to complex logic.
- The importance of real-time.
- The system needs to provide real-time feedback on user actions.

Future Works: Roadmap to a "Security Lab"

Transitioning from a Controller to a Research Testbed

Feature	Strategic Goal	Feasibility
Auth & Access	Role-based Access Control (RBAC) & TLS.	High
Scene Nesting	Recursive logic for complex automation reuse.	High
Attack Actor	Inject fault events and DoS simulations.	Medium
GenAI Config	GenAI-driven intent understanding & config generation.	Medium
Physical Testbed	Real-world validation on physical hardware (no simulation).	Low (Hard)

Reflection: Analysis of Outcomes

Part 1: What worked well vs. What did not?

What worked well? ✓

- **Strict Onion Architecture:** Strong decoupling allowed changing Infrastructure (e.g., DB) without touching Domain logic.
- **API-First Design:** Defined Contracts (OpenAPI) first, enabling Frontend to develop without blocking on Backend.
- **Mock Server:** Effectively decoupled development progress from unstable hardware availability.

What did NOT work? ✕

- **Integration Timing:** "Big Bang" integration at the end caused hidden bugs (Leading to "Tracer Bullet" idea).
- **Manual Object Mapping:** Tedious conversion between DTOs, Domain Entities, and ORM Models resulted in boilerplate.
- **Over-Engineering:** Initial focus on "Perfect Architecture" detached us from actual User Scenarios.

What would we do differently next time?

Part 2: Hypothetical Corrective Actions

- **Process: "Tracer Bullet" Approach**

Build **vertically**: Implement one full feature (e.g., "Toggle Light") from UI to Hardware in Week 1 to validate the path.

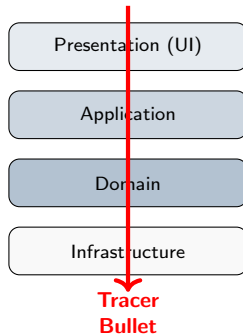
- **Scope: Feature-First, Architecture-Second**

Implement core value (Attack Sim) first, refactor later. Don't start with heavy frameworks.

- **Testing: Real Hardware Earlier**

Mandate testing on simple hardware early (Week 2), rather than 100% Mock Server until the end.

Vertical Slice



Lessons Learned

Part 3: Key Takeaways for our Engineering Career

1. The Cost of Abstraction

Clean Architecture is powerful but expensive. For early-stage demos or MVPs, **YAGNI** (You Aren't Gonna Need It) should outweigh architectural purity.

2. The "Integration Tax"

Integration risks grow exponentially with time. Delaying integration until the end implies that you are creating a "debt" of unverified assumptions.

3. Contracts set you Free

Investing time in a strict **JSON Schema/OpenAPI** at the very beginning was the single best decision we made. It allowed 5 people to work in parallel without blocking.

Thank You!

Questions?